

AI Debate

Aaron Roth

University of Pennsylvania

Fall 2026

Overview

- ▶ How can a weaker judge evaluate answers from stronger systems?

Overview

- ▶ How can a weaker judge evaluate answers from stronger systems?
- ▶ Debate gives an opposing system the job of exposing a mistake that the judge can check.

Overview

- ▶ How can a weaker judge evaluate answers from stronger systems?
- ▶ Debate gives an opposing system the job of exposing a mistake that the judge can check.
- ▶ A carefully designed conversation lets an efficient judge verify the result of a computation that may take exponential time.

Can a Weaker Judge Oversee Stronger Systems?

Even when the information is available, checking an answer may require more reasoning than a human judge can perform (imagine e.g. a 10,000 page mathematical proof)

Can a Weaker Judge Oversee Stronger Systems?

Even when the information is available, checking an answer may require more reasoning than a human judge can perform (imagine e.g. a 10,000 page mathematical proof)

The debate proposal:

1. One AI defends an answer; another tries to expose a mistake.
2. The interaction narrows the disagreement to a claim the human can check.
3. The judge decides who wins; each AI is rewarded for winning the debate.

Can a Weaker Judge Oversee Stronger Systems?

Even when the information is available, checking an answer may require more reasoning than a human judge can perform (imagine e.g. a 10,000 page mathematical proof)

The debate proposal:

1. One AI defends an answer; another tries to expose a mistake.
2. The interaction narrows the disagreement to a claim the human can check.
3. The judge decides who wins; each AI is rewarded for winning the debate.

Can we design the rules so that the defender wins if and only if the answer is correct?

Can a Weaker Judge Oversee Stronger Systems?

Even when the information is available, checking an answer may require more reasoning than a human judge can perform (imagine e.g. a 10,000 page mathematical proof)

The debate proposal:

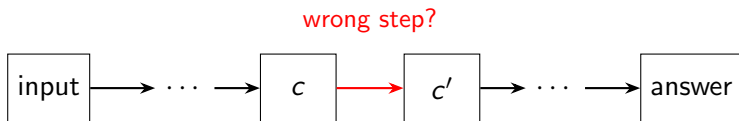
1. One AI defends an answer; another tries to expose a mistake.
2. The interaction narrows the disagreement to a claim the human can check.
3. The judge decides who wins; each AI is rewarded for winning the debate.

Can we design the rules so that the defender wins if and only if the answer is correct?

This is a question about *scalable oversight*: keeping evaluation manageable as the systems being evaluated become more capable than the human overseers.

A Long Computation Can Hide a Small Error

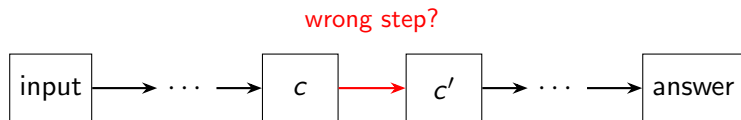
Suppose a system supports its answer with a very long computation or proof.



Here c, c' are consecutive claimed machine states or proof steps.

A Long Computation Can Hide a Small Error

Suppose a system supports its answer with a very long computation or proof.

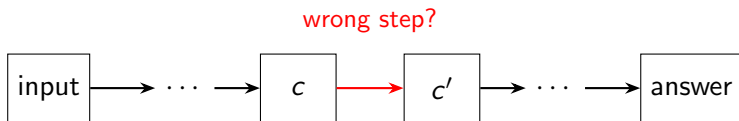


Here c, c' are consecutive claimed machine states or proof steps.

- ▶ The judge can check a specified transition.
- ▶ The judge cannot afford to find the faulty transition by reading the entire computation.

A Long Computation Can Hide a Small Error

Suppose a system supports its answer with a very long computation or proof.



Here c, c' are consecutive claimed machine states or proof steps.

- ▶ The judge can check a specified transition.
- ▶ The judge cannot afford to find the faulty transition by reading the entire computation.

An opponent can do the expensive search. We need rules that force the disagreement to end at a decisive, checkable step.

The Debaters Are Rewarded for Opposite Verdicts

Let $x \in \{0, 1\}^n$ be the input and $f(x) \in \{0, 1\}$ its correct answer.

The Debaters Are Rewarded for Opposite Verdicts

Let $x \in \{0, 1\}^n$ be the input and $f(x) \in \{0, 1\}$ its correct answer.

- ▶ The *proponent* P is assigned answer 1.
- ▶ The *opponent* O is assigned answer 0.
- ▶ Their messages are public. After the conversation, the judge outputs a verdict $J \in \{0, 1\}$.

The Debaters Are Rewarded for Opposite Verdicts

Let $x \in \{0, 1\}^n$ be the input and $f(x) \in \{0, 1\}$ its correct answer.

- ▶ The *proponent* P is assigned answer 1.
- ▶ The *opponent* O is assigned answer 0.
- ▶ Their messages are public. After the conversation, the judge outputs a verdict $J \in \{0, 1\}$.

The proponent receives payoff J ; the opponent receives $-J$. Thus P maximizes the probability of verdict 1, and O minimizes it.

The Debaters Are Rewarded for Opposite Verdicts

Let $x \in \{0, 1\}^n$ be the input and $f(x) \in \{0, 1\}$ its correct answer.

- ▶ The *proponent* P is assigned answer 1.
- ▶ The *opponent* O is assigned answer 0.
- ▶ Their messages are public. After the conversation, the judge outputs a verdict $J \in \{0, 1\}$.

The proponent receives payoff J ; the opponent receives $-J$. Thus P maximizes the probability of verdict 1, and O minimizes it.

The payoff is the judge's verdict, not agreement with $f(x)$. We must design the protocol so that the correct side can force the correct verdict.

What Must a Correct Debate Protocol Guarantee?

The *protocol* specifies legal messages, turn order, and the judge's decision. A *strategy* specifies what to say after each possible history of messages, so it can respond to what the other player has said.

What Must a Correct Debate Protocol Guarantee?

The *protocol* specifies legal messages, turn order, and the judge's decision. A *strategy* specifies what to say after each possible history of messages, so it can respond to what the other player has said.

Perfect completeness

If $f(x) = 1$, the proponent has one strategy that forces verdict 1 against every opponent strategy.

What Must a Correct Debate Protocol Guarantee?

The *protocol* specifies legal messages, turn order, and the judge's decision. A *strategy* specifies what to say after each possible history of messages, so it can respond to what the other player has said.

Perfect completeness

If $f(x) = 1$, the proponent has one strategy that forces verdict 1 against every opponent strategy.

Perfect soundness

If $f(x) = 0$, the opponent has one strategy that forces verdict 0 against every proponent strategy.

What Must a Correct Debate Protocol Guarantee?

The *protocol* specifies legal messages, turn order, and the judge's decision. A *strategy* specifies what to say after each possible history of messages, so it can respond to what the other player has said.

Perfect completeness

If $f(x) = 1$, the proponent has one strategy that forces verdict 1 against every opponent strategy.

Perfect soundness

If $f(x) = 0$, the opponent has one strategy that forces verdict 0 against every proponent strategy.

The judge need not know in advance which side can win.

Polynomial Memory Can Support a Long Computation

Fix a deterministic Turing machine M , our model of a program, that halts on every input x and outputs $f(x)$.

Polynomial Memory Can Support a Long Computation

Fix a deterministic Turing machine M , our model of a program, that halts on every input x and outputs $f(x)$.

Definition

A binary decision problem is in PSPACE if some such machine solves it using at most polynomially many bits of memory in the input length n .

Polynomial Memory Can Support a Long Computation

Fix a deterministic Turing machine M , our model of a program, that halts on every input x and outputs $f(x)$.

Definition

A binary decision problem is in PSPACE if some such machine solves it using at most polynomially many bits of memory in the input length n .

For example, a program can enumerate all 2^n assignments to n Boolean variables while keeping only the current assignment and a small amount of working memory.

Closer to our concerns: *does a deterministic policy reach an unsafe state within a specified exponential number of steps?* This also uses only polynomial memory if the policy and simulator do.

Polynomial Memory Can Support a Long Computation

Fix a deterministic Turing machine M , our model of a program, that halts on every input x and outputs $f(x)$.

Definition

A binary decision problem is in PSPACE if some such machine solves it using at most polynomially many bits of memory in the input length n .

For example, a program can enumerate all 2^n assignments to n Boolean variables while keeping only the current assignment and a small amount of working memory.

Closer to our concerns: *does a deterministic policy reach an unsafe state within a specified exponential number of steps?* This also uses only polynomial memory if the policy and simulator do.

Polynomial memory does *not* require polynomial running time. Our judge will have polynomial running time, not just polynomial memory.

An Efficient Judge Can Verify Any PSPACE Answer

Theorem

Fix a decision problem f in PSPACE. There is a single public debate protocol, specified by algorithms, that verifies $f(x)$ for every input x with perfect completeness and soundness. Communication length and judge time are polynomial in $|x|$.

An Efficient Judge Can Verify Any PSPACE Answer

Theorem

Fix a decision problem f in PSPACE. There is a single public debate protocol, specified by algorithms, that verifies $f(x)$ for every input x with perfect completeness and soundness.

Communication length and judge time are polynomial in $|x|$.

The protocol may depend on the problem f , but its algorithmic rules are fixed across all inputs and input lengths. This is a *uniform* guarantee.

Only the judge and communication must be efficient; the debaters may do much more computation.

An Efficient Judge Can Verify Any PSPACE Answer

Theorem

Fix a decision problem f in PSPACE. There is a single public debate protocol, specified by algorithms, that verifies $f(x)$ for every input x with perfect completeness and soundness.

Communication length and judge time are polynomial in $|x|$.

The protocol may depend on the problem f , but its algorithmic rules are fixed across all inputs and input lengths. This is a *uniform* guarantee.

Only the judge and communication must be efficient; the debaters may do much more computation.

Proof plan:

1. Express the answer as a claim about the endpoint of a long computation.
2. Let the debaters repeatedly halve that claim until the judge can check a single step.

A Configuration Is a Complete Machine Snapshot

For fixed input x , a *configuration* of the Turing machine M records everything needed to determine the next step: tape contents, control state, and tape-head positions.

A Configuration Is a Complete Machine Snapshot

For fixed input x , a *configuration* of the Turing machine M records everything needed to determine the next step: tape contents, control state, and tape-head positions. Encode each configuration by $m = m(n)$ bits, where m is polynomial in n . Write

$$\begin{aligned} M(c) &= \text{the configuration after one step from } c, \\ M^t(c) &= \text{the configuration after } t \text{ steps from } c. \end{aligned}$$

The input x is fixed and available to this transition rule.

A Configuration Is a Complete Machine Snapshot

For fixed input x , a *configuration* of the Turing machine M records everything needed to determine the next step: tape contents, control state, and tape-head positions. Encode each configuration by $m = m(n)$ bits, where m is polynomial in n . Write

$$\begin{aligned} M(c) &= \text{the configuration after one step from } c, \\ M^t(c) &= \text{the configuration after } t \text{ steps from } c. \end{aligned}$$

The input x is fixed and available to this transition rule. The judge can recognize valid encodings and compute $M(c)$ in polynomial time. Computing $M^t(c)$ by simulation may take time proportional to t .

Make the Final Configuration Known

We want answer 1 to mean that the computation reaches one known configuration. Two harmless modifications to M arrange this.

Make the Final Configuration Known

We want answer 1 to mean that the computation reaches one known configuration. Two harmless modifications to M arrange this.

A unique ending

Before halting, M clears its work memory and resets its head positions, then enters c_{acc} for answer 1 or c_{rej} for answer 0.

Make the Final Configuration Known

We want answer 1 to mean that the computation reaches one known configuration. Two harmless modifications to M arrange this.

A unique ending

Before halting, M clears its work memory and resets its head positions, then enters c_{acc} for answer 1 or c_{rej} for answer 0.

A self-loop after halting

Once halted, the configuration repeats forever: $M(c_{\text{acc}}) = c_{\text{acc}}$ and $M(c_{\text{rej}}) = c_{\text{rej}}$.

We can then ask where the machine is after a *fixed* number of steps, even if the program halts earlier.

Make the Final Configuration Known

We want answer 1 to mean that the computation reaches one known configuration. Two harmless modifications to M arrange this.

A unique ending

Before halting, M clears its work memory and resets its head positions, then enters c_{acc} for answer 1 or c_{rej} for answer 0.

A self-loop after halting

Once halted, the configuration repeats forever: $M(c_{\text{acc}}) = c_{\text{acc}}$ and $M(c_{\text{rej}}) = c_{\text{rej}}$.

We can then ask where the machine is after a *fixed* number of steps, even if the program halts earlier.

Both changes use only polynomially more memory, so the modified machine's configurations still have polynomially many bits. From now on, m denotes that (possibly larger) number of bits.

Exponentially Many Steps Always Suffice

There are at most 2^m configurations.

Exponentially Many Steps Always Suffice

There are at most 2^m configurations.

Before halting, a deterministic computation cannot repeat a configuration: it would then cycle forever, but M halts on every input. So M halts within 2^m steps, and then stays put.

Exponentially Many Steps Always Suffice

There are at most 2^m configurations.

Before halting, a deterministic computation cannot repeat a configuration: it would then cycle forever, but M halts on every input. So M halts within 2^m steps, and then stays put.

With c_0 the initial configuration and $T := 2^m$,

$$f(x) = 1 \iff M^T(c_0) = c_{\text{acc}}.$$

Exponentially Many Steps Always Suffice

There are at most 2^m configurations.

Before halting, a deterministic computation cannot repeat a configuration: it would then cycle forever, but M halts on every input. So M halts within 2^m steps, and then stays put.

With c_0 the initial configuration and $T := 2^m$,

$$f(x) = 1 \iff M^T(c_0) = c_{\text{acc}}.$$

The judge can construct c_0 , c_{acc} , and m without running the long computation. Checking the claim by step-by-step simulation may require up to $T = 2^m$ steps.

The Entire Dispute Fits in Two Endpoints and a Level

At each stage the public state is (c, c', j) , representing the claim

$$M^{2^j}(c) = c'.$$

The proponent defends this claim; the opponent tries to refute it.

The Entire Dispute Fits in Two Endpoints and a Level

At each stage the public state is (c, c', j) , representing the claim

$$M^{2^j}(c) = c'.$$

The proponent defends this claim; the opponent tries to refute it.
Initially,

$$(c, c', j) = (c_0, c_{\text{acc}}, m).$$

The initial claim is true exactly when $f(x) = 1$.

The Entire Dispute Fits in Two Endpoints and a Level

At each stage the public state is (c, c', j) , representing the claim

$$M^{2^j}(c) = c'.$$

The proponent defends this claim; the opponent tries to refute it. Initially,

$$(c, c', j) = (c_0, c_{\text{acc}}, m).$$

The initial claim is true exactly when $f(x) = 1$.

The judge records the current endpoints and level. Once an endpoint has been fixed, neither player can rewrite it.

The Entire Dispute Fits in Two Endpoints and a Level

At each stage the public state is (c, c', j) , representing the claim

$$M^{2^j}(c) = c'.$$

The proponent defends this claim; the opponent tries to refute it.
Initially,

$$(c, c', j) = (c_0, c_{\text{acc}}, m).$$

The initial claim is true exactly when $f(x) = 1$.

The judge records the current endpoints and level. Once an endpoint has been fixed, neither player can rewrite it.

We do not ask the proponent to transmit the entire 2^m -step trace.

A Midpoint Splits One Claim into Two

Suppose $j > 0$. A proposed midpoint configuration z gives two claims:

$$\underbrace{M^{2^{j-1}}(c) = z}_{\text{left half}}, \quad \underbrace{M^{2^{j-1}}(z) = c'}_{\text{right half}}.$$

A Midpoint Splits One Claim into Two

Suppose $j > 0$. A proposed midpoint configuration z gives two claims:

$$\underbrace{M^{2^{j-1}}(c) = z}_{\text{left half}}, \quad \underbrace{M^{2^{j-1}}(z) = c'}_{\text{right half}}.$$

The parent claim is true if and only if *there exists* a midpoint for which *both* halves are true:

$$M^{2^j}(c) = c' \iff \exists z : M^{2^{j-1}}(c) = z \text{ and } M^{2^{j-1}}(z) = c'.$$

A Midpoint Splits One Claim into Two

Suppose $j > 0$. A proposed midpoint configuration z gives two claims:

$$\underbrace{M^{2^{j-1}}(c) = z}_{\text{left half}}, \quad \underbrace{M^{2^{j-1}}(z) = c'}_{\text{right half}}.$$

The parent claim is true if and only if *there exists* a midpoint for which *both* halves are true:

$$M^{2^j}(c) = c' \iff \exists z : M^{2^{j-1}}(c) = z \text{ and } M^{2^{j-1}}(z) = c'.$$

This suggests the roles: the proponent supplies z , then the opponent chooses which half must be defended. The opponent need not supply a competing computation—only a challenge.

The Recursive Debate Protocol

Public state: (c, c', j) , initially (c_0, c_{acc}, m) .

The Recursive Debate Protocol

Public state: (c, c', j) , initially (c_0, c_{acc}, m) .

1. If $j > 0$, the proponent sends an m -bit configuration z .

The Recursive Debate Protocol

Public state: (c, c', j) , initially (c_0, c_{acc}, m) .

1. If $j > 0$, the proponent sends an m -bit configuration z .
2. After seeing z , the opponent chooses *left* or *right*.

The Recursive Debate Protocol

Public state: (c, c', j) , initially (c_0, c_{acc}, m) .

1. If $j > 0$, the proponent sends an m -bit configuration z .
2. After seeing z , the opponent chooses *left* or *right*.
3. The judge replaces the state by

$$\begin{cases} (c, z, j - 1), & \text{if left is chosen,} \\ (z, c', j - 1), & \text{if right is chosen.} \end{cases}$$

The Recursive Debate Protocol

Public state: (c, c', j) , initially (c_0, c_{acc}, m) .

1. If $j > 0$, the proponent sends an m -bit configuration z .
2. After seeing z , the opponent chooses *left* or *right*.
3. The judge replaces the state by

$$\begin{cases} (c, z, j-1), & \text{if left is chosen,} \\ (z, c', j-1), & \text{if right is chosen.} \end{cases}$$

4. Repeat until $j = 0$. The judge outputs 1 exactly when $M(c) = c'$, and otherwise outputs 0.

The Recursive Debate Protocol

Public state: (c, c', j) , initially (c_0, c_{acc}, m) .

1. If $j > 0$, the proponent sends an m -bit configuration z .
2. After seeing z , the opponent chooses *left* or *right*.
3. The judge replaces the state by

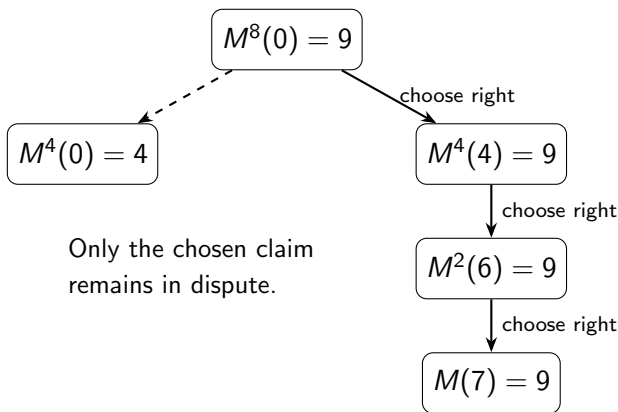
$$\begin{cases} (c, z, j-1), & \text{if left is chosen,} \\ (z, c', j-1), & \text{if right is chosen.} \end{cases}$$

4. Repeat until $j = 0$. The judge outputs 1 exactly when $M(c) = c'$, and otherwise outputs 0.

Messages have fixed length: m bits for a midpoint, one bit for a branch. Invalid configurations lose for the proponent. All updates are public; inherited endpoints stay fixed.

The Opponent Selects a Path, Not an Entire Trace

Let $M(c) = c + 1$. The proponent falsely claims $M^8(0) = 9$.



The proponent supplies midpoints 4, 6, 7; the opponent chooses right each time. Three choices reduce eight steps to one. The counter is easy to compute directly; it illustrates how the protocol works.

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
---------------	----------	-------------------	----------

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
$M^8(0) = 9$	4	$M^4(0) = 4 \checkmark$ / $M^4(4) = 9 \times$	right

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
$M^8(0) = 9$	4	$M^4(0) = 4 \checkmark$ / $M^4(4) = 9 \times$	right
$M^4(4) = 9$	6	$M^2(4) = 6 \checkmark$ / $M^2(6) = 9 \times$	right

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
$M^8(0) = 9$	4	$M^4(0) = 4 \checkmark$ / $M^4(4) = 9 \times$	right
$M^4(4) = 9$	6	$M^2(4) = 6 \checkmark$ / $M^2(6) = 9 \times$	right
$M^2(6) = 9$	7	$M(6) = 7 \checkmark$ / $M(7) = 9 \times$	right

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
$M^8(0) = 9$	4	$M^4(0) = 4 \checkmark$ / $M^4(4) = 9 \times$	right
$M^4(4) = 9$	6	$M^2(4) = 6 \checkmark$ / $M^2(6) = 9 \times$	right
$M^2(6) = 9$	7	$M(6) = 7 \checkmark$ / $M(7) = 9 \times$	right
$M(7) = 9$	judge computes $M(7) = 8 \neq 9$: verdict 0		

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
$M^8(0) = 9$	4	$M^4(0) = 4 \checkmark$ / $M^4(4) = 9 \times$	right
$M^4(4) = 9$	6	$M^2(4) = 6 \checkmark$ / $M^2(6) = 9 \times$	right
$M^2(6) = 9$	7	$M(6) = 7 \checkmark$ / $M(7) = 9 \times$	right
$M(7) = 9$	judge computes $M(7) = 8 \neq 9$: verdict 0		

Here the proponent supplied actual midpoints, leaving the false claim in the right half. If the first midpoint were 5, the opponent would instead choose left, since $M^4(0) = 4 \neq 5$.

A Worked Debate: Where Does the Lie Go?

Continue with $M(c) = c + 1$ and the false claim $M^8(0) = 9$. Why does the opponent keep choosing right?

current claim	midpoint	left / right half	opponent
$M^8(0) = 9$	4	$M^4(0) = 4 \checkmark$ / $M^4(4) = 9 \times$	right
$M^4(4) = 9$	6	$M^2(4) = 6 \checkmark$ / $M^2(6) = 9 \times$	right
$M^2(6) = 9$	7	$M(6) = 7 \checkmark$ / $M(7) = 9 \times$	right
$M(7) = 9$	judge computes $M(7) = 8 \neq 9$: verdict 0		

Here the proponent supplied actual midpoints, leaving the false claim in the right half. If the first midpoint were 5, the opponent would instead choose left, since $M^4(0) = 4 \neq 5$.

At each level, the opponent keeps a false claim in dispute until the judge can check a single step.

Why Not Let the Judge Pick a Half at Random?

For a power-of-two T , let the proponent claim $M^T(0) = T + 1$ and supply actual midpoints. As above, every left half is true and the false claim stays in the right half.

Why Not Let the Judge Pick a Half at Random?

For a power-of-two T , let the proponent claim $M^T(0) = T + 1$ and supply actual midpoints. As above, every left half is true and the false claim stays in the right half.

A judge flipping fair coins catches this lie only by choosing *right* at every one of the $\log_2 T$ levels. If it ever chooses left, the remaining claim is true and the proponent wins from there:

$$\mathbb{P}(\text{lie caught}) = \left(\frac{1}{2}\right)^{\log_2 T} = \frac{1}{T}.$$

Why Not Let the Judge Pick a Half at Random?

For a power-of-two T , let the proponent claim $M^T(0) = T + 1$ and supply actual midpoints. As above, every left half is true and the false claim stays in the right half.

A judge flipping fair coins catches this lie only by choosing *right* at every one of the $\log_2 T$ levels. If it ever chooses left, the remaining claim is true and the proponent wins from there:

$$\mathbb{P}(\text{lie caught}) = \left(\frac{1}{2}\right)^{\log_2 T} = \frac{1}{T}.$$

On this example, random branching is as unlikely to catch the error as checking one uniformly random step.

Why Not Let the Judge Pick a Half at Random?

For a power-of-two T , let the proponent claim $M^T(0) = T + 1$ and supply actual midpoints. As above, every left half is true and the false claim stays in the right half.

A judge flipping fair coins catches this lie only by choosing *right* at every one of the $\log_2 T$ levels. If it ever chooses left, the remaining claim is true and the proponent wins from there:

$$\mathbb{P}(\text{lie caught}) = \left(\frac{1}{2}\right)^{\log_2 T} = \frac{1}{T}.$$

On this example, random branching is as unlikely to catch the error as checking one uniformly random step.

An opponent able to find a false half catches the error with probability 1.

The power comes from the challenger's knowledge,
not just from opposed incentives.

Completeness: Keep Both Halves True

Suppose $f(x) = 1$, so the initial claim is true.

Completeness: Keep Both Halves True

Suppose $f(x) = 1$, so the initial claim is true. At any reached true claim $M^{2^j}(c) = c'$ with $j > 0$, the proponent supplies the actual midpoint

$$z := M^{2^{j-1}}(c).$$

Completeness: Keep Both Halves True

Suppose $f(x) = 1$, so the initial claim is true. At any reached true claim $M^{2^j}(c) = c'$ with $j > 0$, the proponent supplies the actual midpoint

$$z := M^{2^{j-1}}(c).$$

The left half is true by definition of z . The right half is true because

$$M^{2^{j-1}}(z) = M^{2^{j-1}}\left(M^{2^{j-1}}(c)\right) = M^{2^j}(c) = c'.$$

Completeness: Keep Both Halves True

Suppose $f(x) = 1$, so the initial claim is true. At any reached true claim $M^{2^j}(c) = c'$ with $j > 0$, the proponent supplies the actual midpoint

$$z := M^{2^{j-1}}(c).$$

The left half is true by definition of z . The right half is true because

$$M^{2^{j-1}}(z) = M^{2^{j-1}}\left(M^{2^{j-1}}(c)\right) = M^{2^j}(c) = c'.$$

Either choice leaves a true claim. After m choices, the judge checks a true transition and outputs 1.

Completeness: Keep Both Halves True

Suppose $f(x) = 1$, so the initial claim is true. At any reached true claim $M^{2^j}(c) = c'$ with $j > 0$, the proponent supplies the actual midpoint

$$z := M^{2^{j-1}}(c).$$

The left half is true by definition of z . The right half is true because

$$M^{2^{j-1}}(z) = M^{2^{j-1}}\left(M^{2^{j-1}}(c)\right) = M^{2^j}(c) = c'.$$

Either choice leaves a true claim. After m choices, the judge checks a true transition and outputs 1.

One proponent strategy wins against every opponent strategy.

Soundness: Every False Parent Has a False Half

Suppose the current claim is false:

$$M^{2^j}(c) \neq c', \quad j > 0.$$

Fix *any* midpoint z proposed by the proponent.

Soundness: Every False Parent Has a False Half

Suppose the current claim is false:

$$M^{2^j}(c) \neq c', \quad j > 0.$$

Fix *any* midpoint z proposed by the proponent. If both half-claims were true, then

$$M^{2^j}(c) = M^{2^{j-1}}(M^{2^{j-1}}(c)) = M^{2^{j-1}}(z) = c',$$

contradicting the parent claim's falsity.

Soundness: Every False Parent Has a False Half

Suppose the current claim is false:

$$M^{2^j}(c) \neq c', \quad j > 0.$$

Fix *any* midpoint z proposed by the proponent. If both half-claims were true, then

$$M^{2^j}(c) = M^{2^{j-1}}(M^{2^{j-1}}(c)) = M^{2^{j-1}}(z) = c',$$

contradicting the parent claim's falsity.

For every proposed midpoint, at least one half is false.

Soundness: Every False Parent Has a False Half

Suppose the current claim is false:

$$M^{2^j}(c) \neq c', \quad j > 0.$$

Fix *any* midpoint z proposed by the proponent. If both half-claims were true, then

$$M^{2^j}(c) = M^{2^{j-1}}\left(M^{2^{j-1}}(c)\right) = M^{2^{j-1}}(z) = c',$$

contradicting the parent claim's falsity.

For every proposed midpoint, at least one half is false.

The opponent chooses a false half. Finding it may be expensive, but the judge does not have to find it.

Soundness: Keep One False Claim Alive

Suppose $f(x) = 0$, so $M^{2^m}(c_0) \neq c_{\text{acc}}$.

Soundness: Keep One False Claim Alive

Suppose $f(x) = 0$, so $M^{2^m}(c_0) \neq c_{\text{acc}}$. The opponent uses this rule after each proposed midpoint z :

- ▶ Simulate 2^{j-1} steps from c .
- ▶ If the result is not z , choose left.
- ▶ Otherwise choose right, which must be false by the preceding slide.

Soundness: Keep One False Claim Alive

Suppose $f(x) = 0$, so $M^{2^m}(c_0) \neq c_{\text{acc}}$. The opponent uses this rule after each proposed midpoint z :

- ▶ Simulate 2^{j-1} steps from c .
- ▶ If the result is not z , choose left.
- ▶ Otherwise choose right, which must be false by the preceding slide.

The current claim stays false while j decreases from m to 0. At the end,

$$M(c) \neq c',$$

so the judge outputs 0.

Soundness: Keep One False Claim Alive

Suppose $f(x) = 0$, so $M^{2^m}(c_0) \neq c_{\text{acc}}$. The opponent uses this rule after each proposed midpoint z :

- ▶ Simulate 2^{j-1} steps from c .
- ▶ If the result is not z , choose left.
- ▶ Otherwise choose right, which must be false by the preceding slide.

The current claim stays false while j decreases from m to 0. At the end,

$$M(c) \neq c',$$

so the judge outputs 0. This is a *single contingent strategy*, responding to any midpoint the proponent sends. It defeats every proponent strategy, including randomized ones.

What Would Practical Debate Need to Reproduce?

- ▶ **A way to narrow the disagreement.** Our rules fix the endpoints and reduce a global claim to one machine transition. Later messages cannot rewrite earlier claims.

What Would Practical Debate Need to Reproduce?

- ▶ **A way to narrow the disagreement.** Our rules fix the endpoints and reduce a global claim to one machine transition. Later messages cannot rewrite earlier claims.
- ▶ **A reliable local check.** The judge knows the transition rule and checks it correctly even on configurations chosen by an adversary.

What Would Practical Debate Need to Reproduce?

- ▶ **A way to narrow the disagreement.** Our rules fix the endpoints and reduce a global claim to one machine transition. Later messages cannot rewrite earlier claims.
- ▶ **A reliable local check.** The judge knows the transition rule and checks it correctly even on configurations chosen by an adversary.
- ▶ **Debaters able to find winning moves.** The strategies in our proof may do exponential computation. Equilibrium assumes that such deviations are available. If the honest side cannot efficiently locate the false half, a dishonest side can hide its error: the *obfuscated-arguments* problem.

What Would Practical Debate Need to Reproduce?

- ▶ **A way to narrow the disagreement.** Our rules fix the endpoints and reduce a global claim to one machine transition. Later messages cannot rewrite earlier claims.
- ▶ **A reliable local check.** The judge knows the transition rule and checks it correctly even on configurations chosen by an adversary.
- ▶ **Debaters able to find winning moves.** The strategies in our proof may do exponential computation. Equilibrium assumes that such deviations are available. If the honest side cannot efficiently locate the false half, a dishonest side can hide its error: the *obfuscated-arguments* problem.

For natural-language debate, finding an analogous protocol, making its local checks reliable for a human judge, and training capable debaters remain additional tasks.

Summary

- ▶ Debate can let a weaker judge evaluate the work of stronger systems by checking a small, decisive part of the argument.

Summary

- ▶ Debate can let a weaker judge evaluate the work of stronger systems by checking a small, decisive part of the argument.
- ▶ For polynomial-space computations, a short debate and an efficient judge suffice: the correct side can always win.

Summary

- ▶ Debate can let a weaker judge evaluate the work of stronger systems by checking a small, decisive part of the argument.
- ▶ For polynomial-space computations, a short debate and an efficient judge suffice: the correct side can always win.
- ▶ The positive result comes from the protocol's checkable challenges and capable debaters, not from opposed rewards alone.

Thanks!

See you next class!

References

- ▶ Geoffrey Irving, Paul Christiano, and Dario Amodei. *AI Safety via Debate*. arXiv:1805.00899, 2018.
- ▶ Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. *Alternation*. Journal of the ACM, 28(1):114–133, 1981.
- ▶ Beth Barnes and Paul Christiano. *Debate Update: Obfuscated Arguments Problem*. AI Alignment Forum, 2020.
- ▶ Jonah Brown-Cohen, Geoffrey Irving, and Georgios Piliouras. *Scalable AI Safety via Doubly-Efficient Debate*. ICML, 2024.
- ▶ Marie Davidsen Buhl, Jacob Pfau, Benjamin Hilton, and Geoffrey Irving. *An Alignment Safety Case Sketch Based on Debate*. arXiv:2505.03989, 2025.